

Smack Up Your Backup — Security Audit

First Review — v0.7.9h · April 26, 2026

Executive Summary

This first security audit reviews the full source of Smack Up Your Backup (SUYB) v0.7.9h, a desktop Python/Tkinter application that backs up SnapSmack blogs to local disk, Google Drive, Box, and Backblaze B2. The tool handles FTP credentials, SnapSmack admin credentials, cloud service account keys, and OAuth tokens. Scope covers all .py source files: config.py, profile_manager.py, ftp_client.py, backup_engine.py, restore_engine.py, cloud_client.py, and supporting modules.

This audit identified 1 high finding (Google SA private key in repo directory — requires immediate action), 2 medium findings (base64 password obfuscation, FTP TLS cert verification off by default), 2 low findings (Drive API query injection vector, unconditional debug log), and 3 info items. No authentication bypass, remote code execution, or privilege escalation vulnerabilities were found. The tool runs entirely on the user's own machine against their own servers, which limits the attack surface considerably.

#	File	Severity	Finding	Status
1	tools/smack-up-your-backup/	HIGH	Google SA private key in repo directory, not in .gitignore	OPEN
2	profile_manager.py	MEDIUM	FTP and admin passwords stored as base64 (not encrypted)	OPEN
3	ftp_client.py	MEDIUM	FTP TLS cert verification disabled by default (CERT_NONE)	OPEN
4	cloud_client.py DriveClient.list_files()	LOW	name_filter unsanitized in Drive API query string	OPEN
5	cloud_client.py DriveClient.list_files()	LOW	Debug log written unconditionally to exe directory	OPEN
6	cloud_client.py	INFO	OAuth tokens stored as JSON next to credentials file	PASS
7	backup_engine.py	INFO	Broad MIME type acceptance on SQL dump endpoint	PASS
8	backup_engine.py	INFO	In-memory credential retention for session relogin	PASS

Findings

1. Google Service Account Key in Repository Directory

HIGH

OPEN

tools/smack-up-your-backup/suyb-drive-key-5e7a5909f75e.json

A Google service account private key (RSA, 2048-bit) is present in the SUYB tool directory at *tools/smack-up-your-backup/suyb-drive-key-5e7a5909f75e.json*. The file contains **project_id**, **private_key_id**, and a full PEM-encoded private key granting Drive access to the *snapsmack-backups* GCP project. The directory's *.gitignore*

only excludes `*.pyc` and `__pycache__` — the key file is not excluded.

If this file has been committed to git at any point, the key is in git history and must be treated as compromised regardless of whether the file is later removed or `.gitignore`. A key in git history is equivalent to a published secret for any repository that has ever been pushed to a remote.

Immediate action required:

1. Rotate the key now: Google Cloud Console → IAM & Admin → Service Accounts → Keys → revoke key ID **5e7a5909f75e**. Generate a new key and store it at `C:\SmackUpYourBackup\` (the location already documented in CLAUDE.md). Never copy it into the repo directory.
2. Check git history: `git log --all --full-history -- tools/smack-up-your-backup/suyb-drive-key-*.json`. If the file was ever committed, rewrite history with `git filter-repo` to expunge it from all commits, then force-push to all remotes.
3. Add the following patterns to `tools/smack-up-your-backup/.gitignore`:

```
*-drive-key-*.json *_token.json *.json.bak
```

2. FTP and Admin Passwords Stored as Base64 (Not Encrypted)

MEDIUM

OPEN

`profile_manager.py _obfuscate() / _deobfuscate()`

Profile JSON files store the FTP password and SnapSmack admin password as base64-encoded strings (`ftp_pass_enc`, `snap_admin_pass_enc`). The module docstring explicitly acknowledges this: "Passwords are base64-obfuscated (not encrypted) — matches SYBU convention." Base64 is trivially reversible by anyone who can read the `profiles/` directory.

```
def _obfuscate(plain: str) -> str: return base64.b64encode(plain.encode()).decode()
def _deobfuscate(blob: str) -> str: return base64.b64decode(blob.encode()).decode()
```

Any process with filesystem access to `profiles/` can decode all stored credentials in one line of Python. Credentials at risk include FTP passwords (full server access) and SnapSmack admin passwords (full CMS access). Note that `B2 key_id` and `app_key`, if stored in a sync job config, are not obfuscated at all.

Recommended fix: Use the Windows DPAPI via `win32crypt.CryptProtectData` to encrypt credentials. DPAPI binds the ciphertext to the current Windows user account — the data is useless on any other machine or account, with no user-visible change to the workflow. Migration: detect legacy base64 blobs on first load, re-encrypt with DPAPI, save.

3. FTP TLS Certificate Verification Disabled by Default

MEDIUM

OPEN

`ftp_client.py FTPClient.connect()` · `profile_manager.py ftp_verify_cert` default: `False`

When `ftp_verify_cert` is `False` (the default in `new_profile_template()`), the FTP client creates a context with `ssl.CERT_NONE` and disables hostname checking. TLS is in use, but the server certificate is never validated:

```
ctx = ssl.create_default_context() ctx.check_hostname = False ctx.verify_mode = ssl.CERT_NONE
```

With `CERT_NONE`, a network-adjacent attacker can present any certificate and perform a full MITM attack, intercepting both FTP credentials and all backup data. The code comment notes this is "same as clicking Trust this certificate in FileZilla," but the risk is higher here because FTP passwords and SnapSmack admin passwords are also transmitted over the same connection during authentication.

Recommended fix: Default to standard verification. On first connection failure due to cert mismatch, prompt the user with the certificate fingerprint and offer to trust it permanently (storing the fingerprint for future comparison). This is more secure than blanket `CERT_NONE` while remaining usable on shared hosting.

4. Unsanitized name_filter Injected into Drive API Query String

LOW

OPEN

cloud_client.py DriveClient.list_files() query string construction

The *name_filter* parameter is interpolated directly into the Google Drive API query string without sanitization:

```
if name_filter: query += f" and name contains '{name_filter}'"
```

An unescaped single quote in *name_filter* breaks the Drive query syntax (the API uses a SQL-like query language where single quotes delimit string literals). In the current codebase *name_filter* is always a static zip filename, but a profile named with an apostrophe (e.g. Sean's Blog) would cause backup listing to silently fail or throw a Drive API error.

Fix: Escape single quotes before interpolation: *name_filter.replace("'", "''")*

5. Debug Log Written Unconditionally to Exe Directory

LOW

OPEN

cloud_client.py DriveClient.list_files() inner_dbg() function

An inner function *_dbg()* inside *DriveClient.list_files()* writes every Drive listing operation to *suyb-debug.log* next to the exe, unconditionally, on every user's machine. The log records folder IDs, query strings, page counts, and running file totals.

The log grows without bound (no rotation), is readable by any user with access to the exe directory on a shared machine, and is clearly leftover development instrumentation. This is the only place in the codebase where this pattern appears.

Fix: Remove or gate behind a *[debug] verbose = false* config.ini flag. Should be a one-line delete before the next release.

Items Reviewed — No New Risk Found

OAuth Token Storage

INFO · *cloud_client.py _get_drive_service()*

OAuth tokens are stored as **_token.json* next to the credentials file, protected only by filesystem permissions. This matches Google's own sample code and is standard practice for desktop OAuth apps. Tokens use the narrowly-scoped *drive.file* scope (can only access files the app itself created). Readonly tokens use a separate cache file and scope. No new risk.

Broad MIME Type Acceptance on SQL Dump Endpoint

INFO · *backup_engine.py SnapSmackSession.download_sql_dump()*

The SQL dump download accepts *application/sql* and the broad *text/** MIME type to accommodate shared hosting providers that serve .sql files as plain text. The connection is authenticated (session cookie), the endpoint is SnapSmack-controlled, and the dump is stored locally for inspection only (never re-executed). No new risk.

In-Memory Credential Retention for Session Relogin

INFO · *backup_engine.py SnapSmackSession*

SnapSmackSession stores plaintext admin credentials in instance variables to support automatic session relogin during long backup runs. This is a necessary design trade-off for an unattended tool. Credentials are never written to disk by this class and are held only for the BackupEngine object's lifetime. No new exposure beyond Finding 2.

Overall Assessment

SUYB's attack surface is inherently limited: it is a local desktop application that communicates only with the user's own servers and cloud accounts. There is no server-side component, no multi-user session model, and no inbound network listener. The most serious findings are credential storage concerns relevant primarily in the event of local machine compromise.

The SA key in the repo directory (Finding 1) is the only finding requiring immediate action — rotate the key and confirm it is not in git history. The password obfuscation finding (Finding 2) is the most appropriate to address in the next SUYB release: DPAPI is a straightforward change that would bring credential storage up to the standard expected of a Windows desktop application handling server passwords.

Finding	File	Severity	Status
SA key in repo directory — rotate immediately	suyb-drive-key-*.json	HIGH	OPEN
Base64 password obfuscation (not encryption)	profile_manager.py	MEDIUM	OPEN
FTP TLS cert verification off by default	ftp_client.py	MEDIUM	OPEN
Drive API query injection via name_filter	cloud_client.py	LOW	OPEN
Unconditional debug log in DriveClient.list_files()	cloud_client.py	LOW	OPEN

This document is part of SnapSmack's internal security audit series. Previous: 2026-04-26-C-snapsmack-security-audit.pdf